

Guide de Preparation

Questions Jury - 20 Minutes d'Echange

Soutenance Bloc 1 & 2 - RNCP40875

Adam Beloucif & Emilien Morice

Villejuif, 2026

Projet 1

Urban Data Explorer

Projet 2

Maintenance Predictive

Projet 3

L'IA Pero

Document confidentiel - Preparation interne - Ne pas distribuer

Vue d'ensemble - Les 3 projets

La soutenance dure 50 minutes : 30 min de presentation/demo + 20 min d'echange jury. L'evaluation est collective en presentation mais individualisee en notation. Structure imposee : besoin metier -> choix techniques -> realisation -> preuve -> resultats -> limites.

Projet	Stack	Competences	Chiffres cles
Urban Data Explorer	Polars, PostgreSQL, Cassandra, Kafka, FastAPI, Redis	C1.1, C2.4	24 sources, 102 tests
Maintenance Predictive	sklearn, XGBoost, MLP, Streamlit, CodeC	C3.1 a C4.3	F1=0.886, AUC=0.995
L'IA Pero (GenAI)	SBERT all-MiniLM-L6-v2, Gemini, Streamlit	C5.1 a C5.3	384 dims, seuil 0.35

Metriques Maintenance Predictive - a retenir par coeur

Modele	F1	ROC-AUC	CV F1	Score sel.	CO2
LogReg (baseline)	0.747	0.959	0.750	0.746	0.3 mg
Random Forest	0.863	0.992	0.844	0.839	8.2 mg
XGBoost (RETENU)	0.886	0.995	0.886	0.880	6.1 mg
MLP 64-32-16 (DL)	0.836	0.984	0.795	0.790	3.8 mg

CONSEIL GENERAL : Toujours citer le code de competence (C1.1, C4.3...) quand vous repondez. Structure de reponse en 30-45 s : Choix -> Justification -> Preuve dans le code -> Limite.

5 erreurs fatales a eviter

1. Presenter sans relier aux competences -> toujours citer Cx.x.
2. Resultats sans justification -> chaque choix a un "parce que".
3. Oublier les limites -> en donner a chaque projet (recul pro = points).
4. Demo trop longue -> 90 s max chacune, scenariee.
5. Individualisation non preparee -> reponses personnelles, pas "on a fait".

Bloc 1 - Urban Data Explorer

Pitch : Plateforme data complete d'exploration du logement parisien - 24 sources Open Data, architecture medaillon Bronze/Silver/Gold en Polars, PostgreSQL en etoile, Cassandra pour le streaming, Kafka, API FastAPI securisee JWT + quotas, dashboard React/MapLibre charte DSFR.

Questions issues des QA_JURY.md (deja preparees)

C1.1

Pourquoi PostgreSQL ?

Besoin analytique structure (agregats par arrondissement x mois) -> schema en etoile classique, integrite referentielle (FK dim_arrondissement), index sur les axes de requete. Teste en charge : scripts/test_load_postgres.py mesure p95 reel sur les jointures fact/dim.

C1.1

Comment avez-vous garanti l'integrite des donnees ?

Contraintes NOT NULL + FK + cles primaires composees cote PG. Drapeau data_source (real/reference) sur chaque ligne Gold pour la tracabilite. Validation des codes arrondissement par regex normalisee (_normalize_code). Tests pytest sur les transformations.

C1.2

Pourquoi Cassandra en NoSQL ?

Donnees d'evenements semi-structurees a forte velocite (flux urbains). Modelisation query-first : partition par event_type, clustering event_time DESC, TTL 7 jours (la donnee chaude expire seule). Deux patterns d'accès : par type et par arrondissement. PG aurait impose un schema rigide et des purges manuelles.

C1.3

Comment le Data Lake integre-t-il des sources variees ?

24 sources / 8 familles (CSV, GeoJSON, API) atterrissent en Bronze (brut Parquet), normalisees en Silver (codes, geocodage point-in-polygon IRIS), agregees en Gold (datamarts dashboard/timeline). Le format colonneaire Parquet aligne lecture analytique et cout stockage.

C2.1

Quelles contraintes de securite ?

JWT HS256 (secret env var, jamais commite), roles viewer/admin, 401/403 differencies. Quotas par IP fenetre glissante 120 anonyme / 600 authentifie -> 429. CORS restreint a l'origine du front. Donnees Open Data uniquement, pas de donnee personnelle (RGPD by design).

C1.4

Comment l'architecture est-elle scalable ?

Stateless API -> replicable horizontalement. Kafka decouple producteurs/consommateurs. Cassandra scale lineairement par ajout de noeuds. Parquet partitionne par date. Limite assumee : demo mono-noeud (replication_factor 1), on documente le chemin vers un cluster 3 noeuds.

C1.4

Comment avez-vous teste la resilience ?

scripts/test_resilience.py : kill du conteneur Postgres -> l'API bascule en fallback parquet local (reponse degradee mais 200) -> restart -> mesure du temps de recuperation. Healthchecks + restart: unless-stopped + depends_on: service_healthy dans le compose.

C2.4

Comment mesurez-vous la performance des pipelines ?

etl/metrics.py persiste par etape : duree, lignes, octets, lignes/sec dans un parquet de metriques, expose via GET /pipeline/metrics. Optimisations : lru_cache sur le geocodage inverse, Polars (Rust) plutot que pandas, Parquet colonnaire.

C2.2

Micro-batch vs temps reel ?

Les deux : consumer temps reel evenement par evenement -> Cassandra. streaming/microbatch.py agrege en fenetres tumbling 10 s (count, moyenne par type x arrondissement). Le micro-batch lisse la charge d'ecriture et fournit des agregats prêts a servir.

Questions supplementaires possibles

C2.3

Pourquoi Polars plutot que pandas ?

Polars est ecrit en Rust avec execution lazy et parallelisme automatique. Sur nos volumes (24 sources, fichiers jusqu'a quelques centaines de Mo), Polars est environ 10x plus rapide que pandas sur les jointures et agregations. Sans cluster Spark (cout zero), c'est le meilleur rapport perf/complexite pour un ETL local ou Docker.

C2.1

Pourquoi FastAPI et pas Flask ou Django ?

FastAPI genere le Swagger automatiquement (exigence C2.1 demonstrable en demo live), supporte l'async natif (utile pour les appels externes aux APIs ville), valide les schemas avec Pydantic (moins de code de validation manuel). Flask est trop bas niveau pour une API avec auth + quotas. Django est sur-specifie pour une API pure sans ORM metier.

C1.3

Quelle est la difference entre Bronze, Silver et Gold dans votre Data Lake ?

Bronze : donnees brutes telles que recues (CSV, GeoJSON, JSON API), sans transformation, partitionnees par date d'ingestion. Silver : donnees normalisees (codes arrondissement standardises, geocodage IRIS resolu, valeurs manquantes imputees), pret pour la jointure. Gold : agregats metier (prix m2 median/arrondissement/mois, flux urbains par heure) directement consommables par le dashboard et l'API.

C2.3

Comment avez-vous gere les 24 sources heterogenes ?

Chaque source a un connecteur dedie dans etl/external.py (CSV DVF, GeoJSON arrondissements, API Etalab). Le pipeline Bronze ne transforme pas, il enregistre fidelement. La couche Silver applique une normalisation commune : code arrondissement sur 2 chiffres, timestamp ISO 8601, geometrie WGS84. Un drapeau data_source tracabilite quelle source a produit chaque enregistrement.

C2.1

Comment fonctionne votre systeme de quotas exactement ?

Fenetre glissante par adresse IP sur 60 s : 120 requetes max pour les appels non authentifies, 600 pour les appels avec JWT valide. Un token bucket en memoire (dict IP -> compteur + horodatage reset) renvoie 429 Too Many Requests avec l'entete Retry-After quand le seuil est depasse. Limite : pas persistant entre redemarrages -> Redis en production.

C1.4

Quelles sont les limites de votre architecture en production ?

Mono-noeud : Cassandra avec replication_factor 1, PostgreSQL sans replica standby, Kafka mono-broker. Pas de HA reelle -> une panne du noeud coupe tout. Roadmap documentee : cluster Cassandra 3 noeuds (RF=3), PG avec streaming replication + pgBouncer, Kafka 3 brokers + Schema Registry. Par design pour la demo : ressources limitees, objectif de demontrer les patterns, pas la scalabilite reelle.

Bloc 2 Data Science - Maintenance Predictive Industrielle

Pitch : Systeme intelligent multi-modeles predisant la panne machine sous 24 h a partir de capteurs IoT (vibration, temperature, RPM, pression). 4 modeles compares dont 1 Deep Learning. Dashboard decisonnel Streamlit. API FastAPI. Mesure CO2 CodeCarbon. Dataset : Kaggle - 24 042 lignes x 15 variables.

Questions issues de QA_JURY.md

C3.1

Quelles donnees ? Quels problemes de qualite ?

Kaggle industrial_machine_maintenance : 24 042 lignes, 15 variables capteurs (vibration_rms, temperature_motor, rpm, pressure_level, operating_mode). Problemes : valeurs manquantes (imputation mediane fit sur train uniquement), classes desequilibrees (panne 24 h rare), outliers capteurs (winsorisation justifiee par l'EDA).

C3.3

Pourquoi ces variables ?

EDA -> vibration et temperature moteur sont les plus correlees a la panne (confirme ensuite par feature importance : coherence physique, l'usure mecanique genere vibration et echauffement). On garde les variables redondantes sous surveillance multicolinearite plutot que de supprimer mecaniquement.

C3.1

Comment avez-vous traite le desequilibre des classes ?

Stratified split pour preserver les proportions. class_weight="balanced" sur LogReg et Random Forest. scale_pos_weight sur XGBoost. Metriques adaptees : F1, PR-AUC, Recall prioritaire (un faux negatif = panne non detectee = cout maximal). Ajustement du seuil de decision (models/optimal_threshold.json) : par default 0.5 mais optimise empiriquement sur la courbe Precision/Recall.

C4.2

Quels modeles ? Pourquoi XGBoost en final ?

4 modeles : LogReg (baseline interpretable), Random Forest, XGBoost, MLP 64-32-16 (DL impose). XGBoost retenu : F1 0.886, ROC-AUC 0.995, mais surtout score de selection = $F1 - 0.5 \times \sigma(F1 \text{ CV})$ -> meilleur compromis performance/stabilite. Le MLP fait moins bien : dataset tabulaire de taille moyenne, le gradient boosting reste l'etat de l'art.

C4.2

Comment avez-vous evite le surapprentissage ?

Pipelines sklearn : preprocessing fit uniquement sur train (ADR anti-data-leakage dedie). Cross-validation 5 folds stratifiee. Early stopping MLP (patience=10 sur val_loss). Regularisation : alpha MLP, subsample/colsample XGBoost. Comparaison systematique train/test scores pour detecter l'overfit.

C4.3

Et l'ecoresponsabilite ? (explicite dans C4.3)

CodeCarbon mesure le CO2 par entraînement : XGBoost 6.1 mg vs RF 8.2 mg pour de meilleures perfs -> retenu aussi sur ce critere. La LogReg a 0.3 mg reste pertinente si la contrainte carbone domine. C'est explicitement dans le referentiel C4.3 : a mentionner devant le jury.

C3.2

A qui s'adresse le dashboard ?

Responsable maintenance : KPI flotte, simulation de scenario machine (sliders capteurs), prediction temps reel avec probabilite, top variables influentes en langage metier ("vibration au-dessus de X -> risque eleve"). Distinct des visuels EDA du rapport (consigne explicite du sujet).

C4.3

Limites du modele ?

Donnees simulees (pas de bruit capteur reel). Pas de dimension temporelle exploitee (un LSTM sur sequences serait la suite). Derive possible en production -> monitoring et reentrainement periodique necessaires. Un seul dataset Kaggle : generalisation non validee sur d'autres types de machines.

Questions supplementaires possibles

C3.1

Pourquoi avoir choisi ce dataset Kaggle ?

Criteres : donnees de capteurs industriels reels (format IoT typique), suffisamment grand (24 042 lignes, pas de sur-ajustement trivial), variable cible claire (failure_within_24h binaire), desequilibre de classes present (realiste, force l'apprentissage des techniques adaptees). Dataset open source, reproductible par le jury.

C3.1

Expliquez votre pipeline sklearn et pourquoi le preprocessing est fit uniquement sur le train set.

Pipeline sklearn encapsule : Imputer (mediane) -> StandardScaler -> modele. Le fit() est appele une seule fois sur X_train. Lors du predict(), le transform() utilise les parametres appris sur train. Si on faisait fit sur tout le dataset, les statistiques de normalisation incorporeraient de l'information du jeu de test (data leakage) et les metriques seraient optimistes. Documente dans notre ADR anti-data-leakage.

C4.2

Qu'est-ce que l'early stopping et pourquoi l'avez-vous utilise sur le MLP ?

L'early stopping interrompt l'entrainement quand la perte sur le jeu de validation ne s'ameliore plus pendant N epochs (patience=10 ici). Sans early stopping, le MLP continuerait a apprendre le bruit du train set (overfit) et degraderait ses performances sur les donnees inedites. C'est une forme de regularisation implicite, complementaire a la regularisation L2 (alpha).

C4.3

Comment CodeCarbon mesure-t-il les emissions CO2 ?

CodeCarbon echantillonne la consommation CPU/GPU/RAM en temps reel via les APIs systeme (Intel RAPL, NVIDIA NVML). Il multiplie l'energie consommee (kWh) par le facteur d'emission carbone du reseau electrique local (gCO2eq/kWh), recupere via un mapping pays/region. Notre mesure (0.3 a 8.2 mg selon le modele) est une estimation : la machine de calcul n'etait pas dediee, la mesure inclut le fond de l'OS.

C4.3

Pourquoi avoir choisi $F1 - 0.5 \cdot \sigma(F1_{CV})$ comme score de selection ?

F1 seul ne capture pas la stabilite entre les folds. Un modele avec $F1=0.90$ mais $\sigma=0.15$ est moins fiable qu'un modele $F1=0.88$ avec $\sigma=0.02$. La penalite $0.5 \cdot \sigma$ est un compromis : elle penalise la variance sans ignorer la performance moyenne. XGBoost ressort gagnant sur les deux criteres (F1 et stabilite), ce qui renforce la confiance dans le choix.

C4.3

Quelle est la difference entre ROC-AUC et PR-AUC ? Quand preferer l'une ou l'autre ?

ROC-AUC mesure la capacite a distinguer les classes sur toutes les paires (taux vrai positif vs taux faux positif). Elle est optimiste en cas de desequilibre car le grand nombre de vrais negatifs tire le TPR vers le haut. PR-AUC (Precision-Recall AUC) est plus severe : elle mesure la precision a chaque seuil de rappel, penalisant les faux positifs sur la classe minoritaire. En maintenance predictive avec classes desequilibrees, PR-AUC est le critere principal : on veut detecter les pannes sans noyer le technicien sous les fausses alarmes.

C4.3

Comment detecteriez-vous une derive du modele en production ?

Deux types de derive : derive des donnees (data drift) et derive des predictions. Pour la data drift : monitorer les distributions des capteurs (test KS ou PSI entre la fenetre courante et la distribution d'entrainement). Pour la derive des predictions : mesurer le taux de pannes predites vs pannes reelles sur une fenetre glissante. Outil : Evidently AI ou Great Expectations. Seuil de reentrainement : quand le PSI > 0.2 ou le F1 glissant chute de 5 points.

C3.1

Comment avez-vous evite le data leakage ?

Pipeline sklearn : fit uniquement sur X_{train} , jamais sur X_{val} ou X_{test} . Pas de feature engineering dependant de la cible (pas de target encoding naif). Split chronologique envisage mais non applique : le dataset est simule (pas de dimension temporelle reelle). Documente dans l'ADR anti-data-leakage du repo.

Bloc 2 IA Generative - L'IA Pero (Cocktails)

Pitch : Moteur de recommandation de cocktails par IA semantique. SBERT all-MiniLM-L6-v2 (embeddings 384 dims), similarite cosinus, guardrail semantique seuil 0.35, RAG + Google Gemini avec cache MD5. Interface Streamlit Speakeasy. Thematique alternative validee par l'Annexe I du sujet.

Questions issues de QA_JURY.md

C5.1

Pourquoi ce cas d'usage ?

Recommandation par preferences exprimees en langage naturel : les filtres classiques echouent sur "frais et fruite avec une touche tropicale". Thematique alternative validee par l'Annexe I du sujet. La GenAI est adaptee car il faut comprendre (semantique) ET produire (recette personnalisee).

C5.2

Quel modele ? Pourquoi ?

SBERT all-MiniLM-L6-v2 local : gratuit, rapide, 384 dims, suffisant pour la similarite de phrases courtes, pas besoin d'un gros modele pour le retrieval. Gemini free-tier pour la generation uniquement (RAG : le contexte recupere borne la generation).

C5.3

Comment evaluez-vous la qualite des reponses ?

Trois niveaux : (1) guardrail semantique seuil 0.35 teste sur jeux de requetes hors-domaine, (2) tableau requete -> attendu -> score de similarite -> decision, (3) revue humaine des generations (coherence ingredients/recette). Performances dans le rapport (objectif < 3 s de reponse tenu).

C5.3

Quels parametres avez-vous ajustes ?

Seuil de pertinence du guardrail (0.35 : arbitrage faux rejets / hors-sujets). Nombre de resultats Top-N. Temperature de generation Gemini. Structure du prompt RAG (contexte ingredients + profil gustatif injectes).

C5.3

Quels risques ?

Hallucinations (mitigue par RAG + cache). Dependance API externe (mitigue par fallback et cache MD5). Biais du dataset cocktails. Usage responsable de l'alcool mentionne comme limite ethique. Cout : appels strictement limites + caching = conforme a la contrainte free-tier du sujet.

C5.2

Comment l'industrialiser ?

DB vectorielle (pgvector/FAISS) au lieu de la matrice en memoire. Fine-tuning SBERT sur le domaine. Monitoring des prompts/reponses. Journalisation des appels GenAI. Validation humaine en boucle.

C5.2

Pourquoi SBERT et pas un embedding propriétaire (OpenAI, Cohere) ?

Choix cout/autonomie : SBERT all-MiniLM-L6-v2 tourne en local, aucun appel reseau sur le retrieval, 0 latence API, 0 cout par token. Pour la similarite de phrases courtes sur un referentiel ferme (cocktails), 384 dims suffisent : verifie empiriquement (Top-5 coherents sur les requetes de test). Un embedding propriétaire aurait cree une dependance externe sur le chemin critique du retrieval.

C5.3

Que se passe-t-il quand un utilisateur demande hors-domaine ?

Le guardrail calcule la similarite cosinus entre la requete et l'ensemble du referentiel cocktails. Si le max est inferieur a 0.35, la requete est rejetee proprement avec un message explicite avant tout appel Gemini. Teste sur : "repare mon velo", requetes vides, injections de prompt -> tous tombent sous le seuil. Risque residuel : une requete semantiquement proche d'un cocktail par accident peut passer -> limite documentee.

C5.3

Pourquoi le seuil a 0.35 precisement ?

Calibration empirique sur 30 requetes labelisees manuellement (15 in-domain, 15 hors-domain). Courbe Recall/Precision du guardrail en variant le seuil de 0.1 a 0.6. A 0.35 on maximise le F1 du guardrail (rejeter le hors-domaine sans bloquer le in-domain). Documente dans le rapport, reproductible via le script d'evaluation.

Questions supplementaires possibles

C5.2

Qu'est-ce qu'un embedding vectoriel et pourquoi SBERT ?

Un embedding est une representation d'un texte sous forme de vecteur numerique dans un espace semantique (ici 384 dimensions). Deux textes semantiquement proches ont des vecteurs proches (cosinus eleve). SBERT (Sentence-BERT) est un modele transformer fine-tune pour produire des embeddings de phrases complets (pas juste des tokens), ce qui le rend bien adapte a la comparaison de descriptions de cocktails.

C5.2

Expliquez la similarite cosinus avec un exemple concret.

La similarite cosinus mesure l'angle entre deux vecteurs, independamment de leur norme. Exemple : vecteur("frais et fruity") et vecteur("Mojito : menthe, citron vert, sucre") ont un angle faible (similaires) -> cosinus proche de 1. vecteur("repare mon velo") et vecteur("Mojito") ont un angle eleve -> cosinus proche de 0. Formule : $\cos(\theta) = \frac{A \cdot B}{\|A\| \times \|B\|}$. Avantage : insensible a la longueur des textes, ce qui compte c'est la direction semantique.

C5.2

Qu'est-ce que le RAG et pourquoi l'avez-vous utilise ?

RAG = Retrieval-Augmented Generation. Au lieu de laisser le LLM generer librement (risque d'hallucination), on recupere (retrieval) les cocktails les plus pertinents via SBERT + cosinus, on les injecte dans le prompt (augmented context), puis le LLM (Gemini) genere une reponse ancree dans ce contexte. Avantages : moins d'hallucinations, reponses factuelles, pas besoin de fine-tuner le LLM sur notre domaine.

C5.2

Comment fonctionne votre cache MD5 ?

A chaque appel Gemini, on calcule un hash MD5 de la requete normalisee + contexte recupere. Si ce hash existe dans le cache (fichier JSON local), on retourne la reponse deja generee sans appel API. Avantages : cout zero sur les requetes repetees, latence < 100 ms au lieu de 2-3 s, conforme a la contrainte free-tier (quotas Gemini). Limite : le cache ne prend pas en compte le contexte utilisateur precedent.

C5.3

Quels biais votre systeme peut-il introduire ?

Biais de dataset : le referentiel de cocktails (cocktails.csv) sur-represente les cocktails anglo-saxons et sous-represente les boissons non-alcoolisees. Biais de representation : SBERT a ete entraine sur un corpus anglais general, pas sur la terminologie de la mixologie. Biais de confirmation : le cache peut retourner une vieille reponse si le referentiel a ete mis a jour. Biais ethique : le systeme ne verifie pas l'age de l'utilisateur (limite documentee).

C5.2

Comment passeriez-vous en production ce systeme ?

Infrastructure : conteneuriser l'app Streamlit + modele SBERT via Docker. Remplacement de la matrice en memoire par FAISS ou pgvector pour le retrieval scalable. Cache Redis au lieu du fichier JSON local. Monitoring : journalisation des requetes/reponses dans une DB pour detecter les derives et alimenter le fine-tuning. CI/CD : tests automatiques du guardrail a chaque mise a jour du referentiel. Conformite : filtre d'age RGPD, disclaimer usage responsable de l'alcool.

Questions d'individualisation - Adam Beloucif

IMPORTANT : L'évaluation est individualisée. Répondez à la première personne. Jamais "on a fait", toujours "j'ai conçu", "j'ai implémenté", "j'ai choisi".

Quelle est votre contribution principale sur ce projet ?

Sur Urban Data Explorer : j'ai conçu et implémenté l'architecture data de bout en bout (pipeline médaille Polars, schéma étoile PostgreSQL avec FK et index, modélisation Cassandra query-first, API FastAPI avec JWT et quotas). Sur Maintenance Predictive : j'ai implémenté les modèles XGBoost et MLP, le pipeline sklearn anti-leakage, la sélection de modèle par score composite et le dashboard Streamlit. Sur l'IA Pero : j'ai conçu le pipeline SBERT, le garde-rail sémantique, le cache MD5 et l'intégration Gemini RAG.

Quelle est la compétence que vous maîtrisez le mieux ?

C2.1 (API sécurisée) et C4.3 (évaluation comparative). Sur C2.1 : j'ai implémenté de A à Z le système JWT avec rôles, les quotas par fenêtre glissante et les codes d'erreur différenciés -> démontrable live en 30 secondes. Sur C4.3 : j'ai conçu le score de sélection $F1 - 0.5 \cdot \sigma$, mesure le CO2 avec CodeCarbon et produit le tableau comparatif des 4 modèles.

Quel choix technique avez-vous personnellement porté et pourquoi ?

J'ai choisi Polars plutôt que pandas/Spark pour l'ETL Urban Data Explorer. Justification : moteur Rust, parallélisme automatique, 10x plus rapide sur nos volumes sans le coût opérationnel d'un cluster Spark. J'ai validé ce choix avec un benchmark rapide sur les 24 sources avant de l'intégrer dans le pipeline médaille.

Quelle difficulté avez-vous rencontrée et comment l'avez-vous résolue ?

Deux difficultés majeures. Premièrement, la jointure spatiale IRIS (point-in-polygon) était trop lente (> 30 s par fichier DVF) -> solution : cache LRU sur les résultats de géocodage et arrondi des coordonnées à 4 décimales (précision 11 m, suffisante pour IRIS). Deuxièmement, le déséquilibre de classes en maintenance prédictive (pannes rares) rendait l'accuracy trompeuse -> solution : class_weight, PR-AUC, ajustement du seuil de décision.

Que referiez-vous différemment ?

J'intégrerais les métriques de pipeline (rows/sec, durée par étape) dès le premier commit, pas en consolidation finale. J'écrirais les tests unitaires en même temps que le code (TDD), pas en fin de sprint. Sur le MLP, j'expérimenterais une architecture avec BatchNorm pour voir si les résultats s'améliorent. Sur l'IA Pero, j'utiliserais FAISS dès le début plutôt qu'une matrice en mémoire.

Réponse si le jury soulève le binôme ia-pero (Amina vs Emilien)

"Le projet IA générative a été mené dans un binôme différent, avec Amina Medjdoub. Emilien et moi présentons nos projets respectifs en commun car nos Blocs 1 et 2 sont liés. Sur l'IA Pero, j'ai personnellement conçu et implémenté le pipeline SBERT, le garde-rail sémantique, le cache MD5, l'intégration Gemini et l'évaluation. Je peux répondre à toutes les questions techniques sur ce projet." -> Ne pas hésiter, ne pas

s'excuser. L'évaluation est individualisée.

Questions pieges et rares

Pourquoi avoir simule les donnees plutot que d'utiliser des donnees reelles ?

Le dataset Kaggle industrial_machine_maintenance est base sur des capteurs reels mais anonymises et legerement biaises par le createur pour le partage public. En contexte pedagogique, l'objectif est de maitriser la methode, pas de resoudre un probleme industriel reel. En production, on utiliserait des donnees de l'OPCUA du systeme SCADA de la machine, avec horodatage et historique de maintenance.

Votre modele est-il reproductible ?

Oui. Chaque notebook fixe la graine aleatoire (random_state=42) pour sklearn et numpy. Le dataset Kaggle est versionne et le lien de telechargement est dans le README. requirements.txt fixe les versions exactes (sklearn==1.4.0, xgboost==2.0.3). Les metriques sont sauvegardees dans reports/03/metrics_summary.json et peuvent etre regenerees avec : python scripts/03_train_models.py.

Comment avez-vous gere la vie privree dans Urban Data Explorer ?

RGPD by design : uniquement des donnees Open Data (DVF, INSEE Filosofi, APIs Etalab). Pas de donnee personnelle : DVF ne contient que des adresses et prix (agregats), pas d'identite d'acheteur. Les logs API ne conservent pas l'adresse IP au-dela de la fenetre de quota (60 s). Pas de cookie de tracking cote front.

Quel serait le cout de votre solution en production ?

Urban Data Explorer : infrastructure Docker sur un VPS 4 vCPU / 8 Go RAM suffisant pour la demo -> ~25 EUR/mois (Hetzner CX31). En scalabilite : cluster Kubernetes managee (GKE Autopilot) -> proportionnel a la charge, ~100-300 EUR/mois. IA Pero : SBERT local = 0 EUR/token. Gemini free-tier = 0 EUR sous les quotas. En prod : Gemini Flash 8B -> ~0.0375 USD/M tokens d'entree.

Qu'est-ce que le principe de moindre privilege et comment l'appliquez-vous ?

Donner a chaque composant uniquement les permissions dont il a besoin, pas plus. Applications dans notre projet : role viewer = lecture seule (GET uniquement), role admin = lecture + ecriture (GET + POST /pipeline/run). L'API ne tourne pas en root dans le conteneur (USER appuser). Le secret JWT est injecte via variable d'environnement, jamais hardcode dans le code.

Comment votre systeme se comporte-t-il en cas d'indisponibilite d'une source externe ?

Urban Data Explorer : chaque ETL a un try/except par source. En cas d'echec, la source est marquee en erreur dans les metriques et les donnees existantes en Bronze sont conservees (pas d'ecrasement). L'API repond avec les donnees Gold existantes (qui peuvent etre perimees) et expose un champ last_updated pour indiquer la fraicheur. IA Pero : si Gemini est indisponible, retour du Top-5 SBERT seul (recommandation sans generation).

Quelle est la difference entre overfitting et underfitting ? Comment les diagnostiquer ?

Overfitting : le modele a appris le bruit du training set, son score train est nettement superieur au score test.
Diagnostic : courbe d'apprentissage (train score vs val score en fonction de la taille du train). Underfitting : le modele est trop simple, scores train et test tous les deux bas. Sur notre projet : XGBoost test F1=0.886 vs train F1=0.91 -> léger overfit acceptable, controle par CV (ecart-type faible).

Annexe - Chiffres a retenir par coeur

Urban Data Explorer

Metrique	Valeur
Nombre de sources	24 sources / 8 familles
Tests pytest	102 tests (branche feat/bloc1-hardening)
Quotas API	120 req/min anonyme - 600 req/min authentifie
Niveaux cartographiques	5 niveaux (ville -> batiment)
Micro-batch fenetre	10 secondes (fenetres tumbling)
Tech stack	Polars, PostgreSQL, Cassandra, Kafka, FastAPI, React, MapLibre

Maintenance Predictive Industrielle

Modele	F1	ROC-AUC	PR-AUC	Score sel.	CO2
LogReg (baseline)	0.747	0.959	0.712	0.746	0.3 mg
Random Forest	0.863	0.992	0.881	0.839	8.2 mg
XGBoost (RETENU)	0.886	0.995	0.921	0.880	6.1 mg
MLP 64-32-16	0.836	0.984	0.857	0.790	3.8 mg

L'IA Pero - Parametres cles

Parametre	Valeur
Modele embedding	SBERT all-MiniLM-L6-v2 (local)
Dimension embedding	384 dimensions
Seuil guardrail	0.35 (calibre sur 30 requetes)
Metrique guardrail	Similarite cosinus max sur tout le referentiel
Generation LLM	Google Gemini free-tier (RAG)
Cache	MD5 (requete + contexte) - JSON local
Latence cible	< 3 secondes de reponse
Top-N recommandations	5 cocktails par requete

Mapping competences RNCP40875

Competence	Projet	Preuve cle

C1.1

Urban Data Explorer

postgres/init.sql - schema etoile, FK, index

C1.2	Urban Data Explorer	cassandra/schema.cql - query-first, TTL, 2 patterns
C1.3	Urban Data Explorer	data/ Bronze/Silver/Gold, etl/metrics.py
C1.4	Urban Data Explorer	docker-compose.yml healthchecks, scripts/test_resilience.py
C2.1	Urban Data Explorer	api/security.py - JWT, roles, quotas 120/600, 429
C2.2	Urban Data Explorer	streaming/producer.py, consumer.py, microbatch.py
C2.3	Urban Data Explorer	etl/processing.py - Polars, jointure IRIS, fusion DVF+INSEE
C2.4	Urban Data Explorer	etl/metrics.py, lru_cache geocodage, Parquet colonnaire
C3.1	Maintenance Predictive	src/preprocessing.py - pipeline sklearn fit-train-only
C3.2	Maintenance Predictive	dashboard/app.py - Streamlit KPI + simulation
C3.3	Maintenance Predictive	scripts/02_edu.py - distributions, correlations, desequilibre
C4.1	Maintenance Predictive	RAPPORT.md strategie IA : cout corrective -> predictive
C4.2	Maintenance Predictive	scripts/03_train_models.py - 4 modeles, hyperparams justifies
C4.3	Maintenance Predictive	reports/03/metrics_summary.json - tableau + CO2 CodeCarbon
C5.1	L'IA Pero	RAPPORT_FINAL.md - personas, scenarios, justification GenAI
C5.2	L'IA Pero	src/recommender.py - SBERT, cosinus, guardrail, cache, Gemini
C5.3	L'IA Pero	tests/test_guardrail.py - tableau requetes/scores/decisions